



AI Agents for Developers Guide / AI Agent Architecture Explained

AI Agent Architecture Explained

AI agent architecture is the engineering discipline that defines how a model, tools, memory, orchestration, and runtime control combine into a coordinated workflow rather than isolated model calls. Most agents begin as a simple idea, a model calling a few tools, then grow into systems complex enough that architecture is what keeps them manageable.

Those decisions shape everything downstream: how reliably the system executes, how observable it is, what permissions it holds, how it scales, the latency it adds, and how it recovers when something breaks.

This is a developer's architecture reference, not a general AI overview. It covers the core components and how they are orchestrated, how agentic workflows execute, the common design patterns, runtime control, and how to choose an architecture for production, each through one lens: how the pieces fit together and what that decides once the system is running. The deeper mechanics of each live in dedicated guides.

On average, developers report that approximately 46% of the code they produce is fully generated by AI agents, 39% is written with AI assistance, and 27% is written entirely manually.

Source: Preliminary findings from the JetBrains Developer Ecosystem Survey 2026 • 15,000+ developers worldwide

Table of content

Why AI agent architecture matters

Core components of AI agent architecture

How AI agent workflows execute

Common AI agent architecture patterns

Runtime control, guardrails, and reliability

Choosing the right AI agent architecture

What this means for developers

Why AI agent architecture matters

Two agents built on the same underlying model but with different orchestration layers behave very differently in production. Architecture is what accounts for the difference: It determines how an agent interprets a goal, selects and sequences actions, manages what it remembers across steps, updates its runtime state, and decides when to stop, retry, or escalate. That is the part you control.

When something goes wrong, the root cause often lies upstream, in the orchestration logic, state management, or permission boundaries. However, model limitations can also be the cause, depending on the workflow. The model is the reasoning engine inside a larger workflow system, not the system itself, so reliability is mostly a property of the design around it. That design starts with the components every production agent shares.

Core components of AI agent architecture

Most production agents combine the same AI agent components: a reasoning model, a set of tools, a memory layer, an orchestration system, and runtime controls, with instructions threading through all of them. What makes it an architecture is how the parts connect, not the parts themselves: A model reasons but needs something to decide when it runs; tools act but need something to bound what

they touch. This section covers those relationships; the components article treats each part in depth.

Models and reasoning systems

The model is the reasoning core, handling planning, generation, classification, and decision support within the workflow. It can be a large language model (LLM) or a smaller specialized model, depending on the task. Architecturally, the defining trait is what the model does not do: It reasons one step at a time but never drives the workflow or calls a tool on its own. That separation is why swapping models changes the quality of individual steps without changing how the system is structured.

Tools and external systems

Tools are what enable an agent to act beyond its context window, reaching APIs, IDEs, terminals, repositories, databases, browsers, or CI/CD systems when permitted. Tool use is what turns reasoning into effects on real systems. The architectural point is that every tool interaction is bounded: It runs against a defined schema, within the permissions it has been granted, and inside the limits the runtime layer enforces. The model proposes a call; the architecture decides whether it is allowed and how it is checked, which is why tool access lives on the runtime-control surface rather than in the model.

Memory and state management

Two different things get called memory, and the architecture treats them differently. Runtime state is

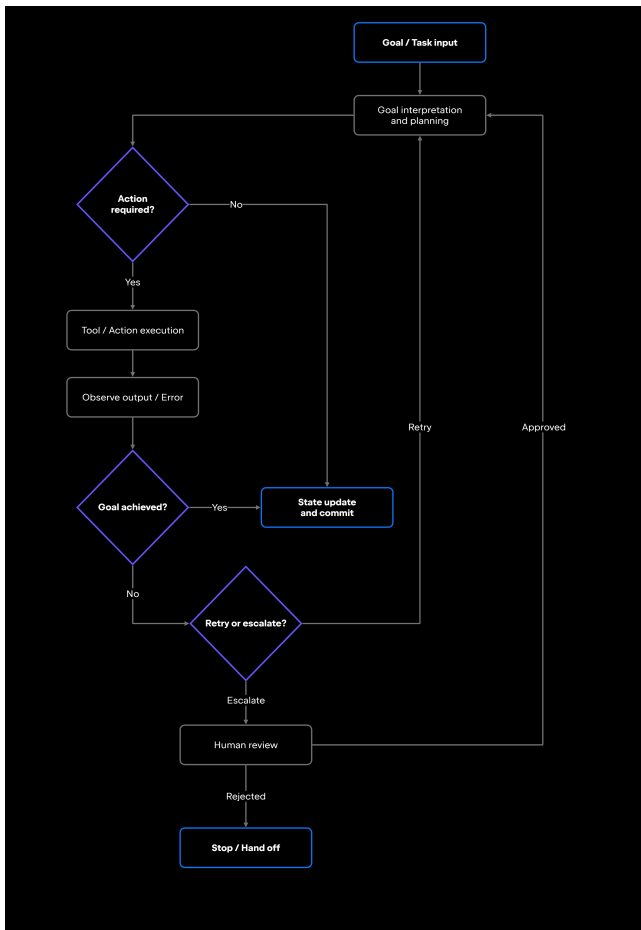
the temporary record of the current run. It includes which steps have been completed, which outputs have been returned, and what remains – and it is discarded when the workflow ends. Persistent memory survives across sessions: conversation history, repository context, task history, and retrieved documents. The trade-off is architectural: Memory improves workflow continuity but increases orchestration complexity because cached context must remain fresh and consistent. The mechanics of each are handled separately in *Memory in AI Agents*.

Orchestration and control flow

Orchestration is the component that makes the others come together into a workflow. It coordinates reasoning, actions, observations, retries, approvals, and state transitions, and it owns the control flow: whether the agent continues, stops, retries, or escalates to a human. Simple agents run a single orchestration loop; more complex ones nest them, a coordinator delegating to specialized sub-agents, each with its own scope. At the organizational level, orchestration can span many agents working against shared workflows under centralized governance. Either way, the components only become an agent once orchestration puts them in motion, which is where the execution loop comes into play.

How AI agent workflows execute

An AI agent workflow is a feedback loop, and the architecture is what drives it. The model contributes one reasoning step at a time; orchestration and runtime control turn those steps into a sequence: Interpret the goal, plan actions, execute tools, observe the outputs, update state, and then continue or stop. The *Agentic Workflows Explained* guide walks the loop end to end; here, the focus is on where the architecture inserts control, because that is where production workflows succeed or fail.



What differs between frameworks is how much of each step they expose for inspection and approval. The three stages below are where those decisions live.

Goal interpretation and planning

A workflow begins by interpreting the objective, identifying missing information, and decomposing it into a sequence of actions. Architecturally, plan quality is set before the model reasons: The context the system supplies, the files, the constraints, and the definition of “done”, all decide whether the plan survives execution or falls apart at the first unexpected result.

Actions and tool execution

Each planned step becomes an action against an external system: an API call, a code execution, a file write. The architectural constraint is that actions run within runtime limits and approval boundaries, not at the model's discretion. A high-stakes call, committing code or touching production, passes through a human-approval gate before it executes. That boundary is where bounded autonomy lives: The agent acts on its own, but a person accepts the consequential results.

Observations, feedback, and state updates

After each action, the loop observes the result – a test output, an error, or a tool response, for instance

– and decides what to do next. This is where the architecture shows: a sound design routes a transient failure to a retry, a semantic failure to a re-plan, and an unrecoverable one to a human, instead of treating every error alike. The state update that follows tracks workflow continuity, what has happened, and what remains, without retraining or changing the model. How that loop is wired, and what each wiring trades off, is what the architecture patterns describe.

Common AI agent architecture patterns

The most common AI agent architecture patterns, ReAct, planner-executor, multiagent, and stateful vs. stateless, are different ways to arrange the same components and loop. Each involves a distinct trade-off across latency, observability, and complexity, and most production systems combine several. AI agent orchestration⁷ is what composes them at the system level, and how they compose often matters more than which one you start with.

ReAct architectures

ReAct (reasoning plus acting) interleaves a reasoning step and an action step at every iteration, looping until the goal is met or a stop condition is met. Architecturally, it keeps planning and acting in a tight loop, making each step easy to trace and well-suited to exploratory work like debugging, issue investigation, or research. JetBrains Junie runs this shape: It reasons, calls a tool, reads the result,

and iterates until the task is done or a stop condition fires, running a failing test, reading the error, and adjusting before it tries again. The cost of that tight loop is added iteration and execution overhead when the plan is already obvious.

Planner-executor architectures

The planner-executor architecture splits the loop into two: The planner decomposes the objective into a task list, and a separate executor carries it out. The payoff is that the plan becomes an inspectable artifact you can review, edit, or reject before any action runs, which improves structure and observability. (*Plan mode*[↗] in JetBrains Air[↗] is one implementation of that review gate.) The exposure is its weakness: An inaccurate plan propagates downstream, so the pattern leans hard on strong goal interpretation up front.

Multiagent architectures

Multiagent systems distribute responsibilities across specialized agents under a coordinator, in arrangements like coordinator-worker, planner-reviewer, or researcher-executor. Multiagent distribution buys parallelism and specialization at the cost of coordination. A failure can surface in any agent or during the handoff between them, which makes these systems the hardest to trace. The coordination patterns themselves are a subject in their own right.

Stateful vs. stateless agent architectures

The split is whether the system carries memory between steps and sessions. Stateless agents rely only on immediate context, which makes them simple, scalable, and easy to reproduce, but unable to sustain long-running work. Stateful agents maintain workflow memory and runtime continuity, trading scalability, added complexity, and harder observability for the ability to span hours-long tasks. Most production systems are hybrid, stateless at the tool-execution level and stateful at the orchestration level. Choosing a pattern is only half the job, though, because keeping it reliable under real conditions takes a separate layer of runtime control.

Runtime control, guardrails, and reliability

Runtime control is the layer that makes an agent safe to deploy. Architecturally, it matters because reliability comes from bounding what the agent can do, not from a model that never errs. The failure modes are predictable and architectural rather than model-level: hallucinated or invalid tool calls, runaway loops that never hit a stop condition, actions that exceed their permissions, and gaps where no one can see what the agent did.

A production architecture answers each at the boundary: Permissions and schema validation govern what the agent may call and on what terms; stopping conditions and bounded execution cap

how long it runs; approval flows, and escalation put a human in front of high-stakes actions and unresolved failures; and monitoring with rollback paths keeps behavior observable and mistakes reversible. Instruction files set the same constraints at the prompt level before a run begins, and organization-wide platforms like JetBrains Central extend that observability across many agents at once.

None of these stops the agent from ever failing; together, they bound the blast radius so it fails safely. How many of these controls a system needs and where they sit are the first things the architecture you choose has to settle, which is exactly the selection problem the next section takes up.

Choosing the right AI agent architecture

Choosing an architecture is a practical decision driven by workflow requirements. The right pattern for a bug investigation differs from the one for a multirepo refactor.

Start with a few questions: How complex is the task decomposition? Does the workflow need to run unattended for long stretches? Which tools does it call, and how sensitive are they? How far does it need to scale? How much observability do you need? What is the latency budget? What is the operational risk if the agent makes a wrong call, and how much human oversight does that risk demand?

Pattern	Strengths	Limitations	Best Fit
ReAct	Traceable reasoning, good for exploration	Higher iteration overhead, slower for well-defined tasks	Debugging, investigation, research
Planner-Executor	Clear task structure, inspectable plans	Brittle if planning phase fails	Code generation, structured task lists
Multiagent	Parallelism, specialization, scalability	Complex coordination, harder to debug	Large-scale workflows, parallel workstreams
Stateless	Simple, reproducible, easy to scale	No continuity across sessions	Short one-shot tasks
Stateful	Long-running workflow continuity	State management overhead	Multistep, session-spanning workflows

These patterns are not mutually exclusive.

Production systems usually combine several rather than choosing just one: a system might pair a multiagent architecture with planner-executor semantics inside each agent, stateful orchestration at the coordinator level, and stateless execution for individual tool calls.

Building an agent is a discipline of its own, but the starting move is always the same: Pick the simplest architecture that meets the requirements and add complexity only as the workflow demands it. A ReAct agent covers a surprising range of work before you need a full multiagent system, and knowing when to make that jump is what separates teams that operate agents from teams whose agents operate.

What this means for developers

Across every section, the same few decisions do the heavy lifting, and they are made early: which tools the agent can reach, how its state is managed, what bounds its autonomy, and where a human stays in the loop. These are architectural choices, and they decide a system's reliability, observability, and safety far more than the model behind them does.

Two habits keep those decisions sound: match the architecture to the workflow rather than defaulting to the most capable pattern, and start with the simplest design that works, adding complexity only as the workflow demands it. Then design for failure containment, since bounding the damage a wrong action can cause matters more than preventing every error.

The agents reaching production today are exactly this: coordinated workflow systems that combine a model with tools, memory, orchestration, and runtime control. Whether you build that scaffolding yourself or adopt a platform like JetBrains Air, the architecture is the part you own, and model quality

becomes a parameter you tune on top of a design you can already trust.

FAQ

How do developers design scalable AI agent architectures?

Scalable AI agent architecture relies on decomposing workflows into independent, bounded subtasks that can run in parallel without shared mutable state. Multiagent architectures with a coordinator and specialized workers scale horizontally, letting you add agents for new task types without rearchitecting the core system. Organization-level platforms handle compute provisioning, cloud execution, and the shared context that horizontal scaling requires. Stateful workflows are harder to scale than stateless ones, so keep state management centralized and explicit.

Which AI agent architecture patterns are easiest to monitor and debug?

Observability is largely set by the pattern. ReAct architectures are among the most observable by design: Every reasoning step produces a readable trace of what the model concluded and why. Planner-executor architectures expose the task decomposition as an inspectable artifact, which makes plan-level failures easy to diagnose. Multiagent systems are the hardest, because failures distribute across agents and handoffs, so the architectural answer is to standardize how every agent reports, letting a single trace read end to end. Whatever the pattern, structured

What architectural tradeoffs exist between simple and complex agent systems?

A simple ReAct agent is easier to debug, cheaper to run, and faster to build, but it doesn't parallelize, struggles with long task horizons, and has limited specialization. A multiagent planner-executor system handles complex, long-running workflows well but adds coordination overhead, more failure points, and higher observability requirements. The inflection point is usually parallelism: when one agent serializing every step becomes the latency bottleneck, or tool access needs splitting for safety, multiagent structure starts to pay off.

What architecture patterns work best for long-running AI workflows?

Long-running AI workflows, such as multifile migrations, hour-long refactoring runs, or overnight CI failure analysis, require stateful orchestration that survives interruption and resumption. Planner-executor architectures fit well because the plan is a durable artifact: If the executor fails midway, it shows where to resume. Asynchronous cloud execution suits this work: Developers fire off a task, come back to review the results, and the agent keeps running in the background. Asynchronous execution with explicit checkpoints and approval gates is the

logging of every step, tool call, and state transition is non-negotiable in production.

What is the difference between stateful and stateless AI agents?

The difference is whether the agent carries memory between runs. Stateless agents start fresh each run, which keeps them simple and easy to scale; stateful agents persist memory across steps and sessions to sustain hours-long tasks, paying state-management overhead for it. As a rule, start stateless and add stateful orchestration only when the workflow needs continuity beyond a single context window.

recommended pattern for workflows that span more than a few minutes.

How do runtime guardrails prevent AI agent failures in production?

Guardrails work by bounding the consequences of an incorrect action rather than trying to prevent the agent from ever making mistakes. The core controls are schema validation, scoped permissions, stopping conditions, human approval for high-stakes actions, structured logging, and rollback paths, all layered so that a failure is contained rather than propagated. Configured together, they allow an agent to continue acting on its own while remaining recoverable when something goes wrong.

Damaso Sanoja

Damaso Sanoja is an engineer who is passionate about helping others make data-driven decisions to achieve their goals. This has motivated him to write numerous articles on the most popular relational databases, customer relationship management systems, enterprise resource planning systems, master data management tools, and, more recently, data warehouse systems used for machine learning and AI projects. You can blame this fixation on data management on his first computer being a Commodore 64 without a floppy disk.

Use AI agents in your development

workflow

Explore JetBrains AI solutions that help you build, use, and scale AI agents across the software development lifecycle.

JetBrains Air

An open system of AI products for developers, teams, and organizations – from coding with agents to automating workflows and governing AI at scale.

[Explore Air](#)

Air in IDEs

Orchestrate AI agents and verify their output directly in JetBrains IDEs, with full control over how changes are reviewed.

[Try Air in IDEs ↗](#)

Air Teams

Automate software delivery workflows, coordinate agentic work across teams, identify bottlenecks affecting delivery.

[Explore Air Teams ↗](#)

Air Governance

Govern AI usage, models, policies, and costs across your organization, including JetBrains and third-party tools.

[Explore Air Governance ↗](#)

Air Context

Give agents shared organizational memory and context that carries across

Air Gateway

Run terminal agents through one JetBrains account, with access, models, and usage

Junie

Plan, code, debug, and automate tasks with a coding agent that works across your

tools, workflows,
and execution
environments.

[Explore Air
Context ↗](#)

governed
centrally.

[Explore Air
Gateway ↗](#)

terminal, IDE,
GitHub, and
GitLab.

[Try Junie ↗](#)

Continue Exploring the AI Agents for Developers Guide

ReAct Agents Explained: How They Work

Learn how ReAct agents combine reasoning, tool use, and feedback loops, where they work best, and how to manage reliability, cost, and latency.

[Read more ↗](#)

AI Agent Orchestration: How It Works

Learn how AI agent orchestration works, from planning and task routing to state management, multi-agent coordination, and reliable workflow execution.

[Read more ↗](#)

Multi-Agent Systems for Developers

Learn how multi-agent systems coordinate AI agents, compare architecture patterns, solve complex workflows, and improve software development.

[Read more ↗](#)